

Exact Subset Sum and Subset Balancing: A Structured Survey from Meet-in-the-Middle to the Representation Frontier

Reproducible Research Note

Abstract—For n integers and a target, exact subset sum has remained at the meet-in-the-middle bound $\mathcal{O}^*(2^{n/2})$ for half a century in the worst case, while the average-case representation technique now reaches $\tilde{\mathcal{O}}(2^{0.291n})$ for hard knapsacks. This survey maps the field. We fix the problem variants (subset sum, subset balancing over a coefficient set C , Equal-Subset-Sum, Pigeonhole ESS, k -SUM, cardinality-constrained, dense, modular), tabulate the classical and modern algorithms with their time, space, model, and regime, and trace three distinct lines: worst-case exact algorithms, average-case/cryptanalytic algorithms, and pseudopolynomial/dense algorithms. We give a detailed account of the representation and dissection technique from its origin through the recent worst-case results, state the remaining open coefficient sets ($\{0, 1\}$, $\{\pm 1\}$, $\{\pm 2\}$) precisely, collect the conditional lower bounds that constrain them, and give a chronology. A companion note reproduces the representation exponents from the primary sources. All claims are sourced; reproduced values are marked as such.

Index Terms—subset sum, subset balancing, meet-in-the-middle, representation technique, dissection, hard knapsacks, exact exponential algorithms, survey

I. INTRODUCTION

A. The problem

Given integers $a_1, \dots, a_n$ and b , the *exact subset sum* decision problem asks whether some $x \in \{0, 1\}^n$ satisfies $\sum_{i=1}^n a_i x_i = b$; the search problem asks for a witness. Entries may be negative, repeated values are distinct occurrences, and the empty subset is allowed. Throughout, $\mathcal{O}^*(\cdot)$ suppresses factors polynomial in n and in the input bit length, and $W = \sum_i |a_i|$ is the magnitude sum. The difficulty is a single constant: the pseudopolynomial and dense regimes are well understood, but the leading exponential exponent for *worst-case* exact subset sum remains $1/2$.

B. Why the exponent is subtle

Three structural facts constrain every approach. (i) If $|a_i| \leq \text{poly}(n)$ then $W \leq \text{poly}(n)$ and the dense dynamic program is polynomial — hardness needs large values. (ii) Completely large, superincreasing values (e.g. powers of two) are also easy, because the target’s binary expansion is the witness. (iii) Hardness therefore concentrates near density $d = n / \log_2 \max_i |a_i| \approx 1$, exactly where the cryptanalytic representation technique lives. Powers of two also show that deduplicating subset sums cannot force a better exponent: they produce 2^n distinct sums while remaining easy.

TABLE I
PROBLEM VARIANTS, COEFFICIENT SETS, AND PROMISES.

Variant	Coefficient set / promise	
Subset Sum	$C = \{0, 1\}$, target b	
Partition	$C = \{-1, 1\}$, target 0	
Equal Subset Sum (ESS)	$C = \{-1, 0, 1\}$, target 0	
Pigeonhole (PESS)	ESS	ESS with $\sum_i a_i < 2^n - 1$ (solution guaranteed)
Subset Balancing k -SUM		constant C (e.g. $[\pm d]$, $[-d : d]$) one element from each of k lists, sum to target
Cardinality-constrained Dense		$C = \{0, 1\}$ with $\sum_i x_i = k$
Modular		target t large; near-linear-time regime equation modulo M (e.g. 2^n)

C. Variants

Different variants behave very differently; Table I fixes notation. A *subset balancing* instance over a coefficient set $C \subset \mathbb{Z}$ asks for a nonzero $c \in C^n$ with $\sum_i c_i a_i = 0$; it generalises subset sum and Partition. Densities are $d = n / \log_2 \max_i |a_i|$; the hard-knapsack regime is $d \approx 1$.

D. Scope and sources

This survey compiles primary sources (arXiv, IACR ePrint, conference proceedings) found in a structured literature pass, together with small dependency-free scripts that reproduce the representation exponents. We separate *cited* results from *reproduced* ones (Section IV). No new algorithm or lower bound is claimed.

II. CLASSICAL EXACT ALGORITHMS

a) *Enumeration and dynamic programming.*: Enumerating all 2^n subsets is $\mathcal{O}(2^n)$. Bellman’s dynamic program (1957) runs in $\mathcal{O}(nt)$ time and space for target t (equivalently $\mathcal{O}(nW)$ for the full range). Both are superseded exponentially; they matter because the pseudopolynomial regime is where dense and small-target instances live (Section VII).

b) *Meet-in-the-middle.*: Horowitz and Sahni [1] split the entries into halves L, R , enumerate the $2^{n/2}$ subset sums of each, sort one, and binary-search for complements $u + v = b$. Cost: $\mathcal{O}^*(2^{n/2})$ time and space, deterministic, witness-producing. A split-correctness lemma (a selection of $L \parallel R$

sums to b iff there are $u \in S(L)$, $v \in S(R)$ with $u + v = b$) makes it exact. This is the baseline.

c) Schroeppe–Shamir.: Schroeppe and Shamir [2] keep $\mathcal{O}^*(2^{n/2})$ time but reduce space to $\mathcal{O}^*(2^{n/4})$. They split into four blocks of $n/4$, and for a guessed middle value σ_M enumerate the pair lists $\{\sigma_{12}\}$, $\{\sigma_{34}\}$ matching modulo $2^{n/4}$, generating the needed pair sums lazily in sorted order with two priority queues instead of materialising the $2^{n/2}$ list. The general trade-off is roughly $T = \mathcal{O}(2^n/S^2)$ for space S .

d) Polynomial-space algorithms.: Lokshtanov and Nederlof [7] solve subset sum and knapsack in pseudopolynomial time and polynomial space via algebraization (inclusion–exclusion and Möbius transforms over the DP table). Bansal, Garg, Nederlof and Vyas [6] give $\mathcal{O}^*(2^{0.86n})$ time in polynomial space. These dominate on space, not time.

III. WORST-CASE IMPROVEMENTS BELOW $2^{n/2}$

a) Polynomial-factor shaving.: Chen, Jin, Randolph and Servedio [3] gave the first improvement over Horowitz–Sahni: $\mathcal{O}(2^{n/2}n^{-\gamma})$ for a constant $\gamma > 0.5023$ in standard word RAM or circuit RAM. The gain is polynomial, not a smaller base; it combines the representation method with bit-packing (Baran–Demaine–Pătraşcu).

b) Space versus time.: Nederlof and Węgrzycki [4] reduce worst-case subset sum to the Orthogonal Vectors problem, obtaining $\mathcal{O}^*(2^{0.5n})$ time and $\mathcal{O}^*(2^{0.249999n})$ space (randomized), with a tight two-way relation between the two problems. Belova, Chukhin, Kulikov and Mihajlin [5] lower the space exponent to 0.246. These improve space only.

IV. THE REPRESENTATION AND DISSECTION TECHNIQUE

This is the engine behind the sub-1/2 exponents in the average case and, recently, several worst-case cases.

A. Two-part representation and the modulus filter

Split a weight- $\ell \approx n/2$ solution as $x = y + z$ with y, z disjoint, each of weight $\approx \ell/2$. Each of the ℓ ones goes to y or z , so x has $N = \binom{\ell}{\ell/2} \approx 2^\ell$ representations. Choose a modulus $M \approx N$ and a random residue R , and enumerate $Y = \{y : a \cdot y \equiv R \pmod{M}\}$ and $Z = \{z : a \cdot z \equiv b - R \pmod{M}\}$. A random R keeps about one representation of the plant; matching Y and Z exactly returns it. If the modular lists could be built in time proportional to their size, the cost would be

$$D(\alpha) = \frac{2}{\alpha^{\alpha/2}(2-\alpha)^{(2-\alpha)/2}} 2^{-\alpha}, \quad \alpha = \ell/n, \quad (1)$$

with $D(1/2) = 2^{0.3113n}$ in the balanced case (HGJ [8], Sec. 4.1). The gap between $D(\alpha)$ and the achievable time is the crux.

B. Four-way decomposition and unbalanced Schroeppe–Shamir

The concrete HGJ algorithm [8] decomposes $b = \sigma_1 + \sigma_2 + \sigma_3 + \sigma_4$ into four weight- $(n/8)$ parts and solves each modular sub-knapsack with the *unbalanced* Schroeppe–Shamir

TABLE II
HGJ SIMPLE-ALGORITHM TRADE-OFFS [8].

β	time exponent	memory exponent
$\rightarrow 1/4$	0.338	0.294
0.2936	0.381	0.250 ($= n/4$)
0.2972	0.385	0.247
$\rightarrow 1/3$	0.421	0.211

algorithm (a per-list construction costing $C(\alpha) = (\alpha^{-\alpha}(1 - \alpha)^{\alpha-1})^{1/2}$). With modulus $M \approx 2^{\beta n}$, $1/4 < \beta < 1/3$:

$$T(\beta) = \max(2^{0.3113n}, 2^{(0.0872+\beta)n}), \quad M(\beta) = 2^{(0.5436-\beta)n}. \quad (2)$$

Table II lists the trade-offs. May and Meurer corrected HGJ’s claimed $2^{0.3113n}$ to $2^{0.337n}$ by accounting for the pre-filter merge cost: when two lists of size L are merged with a modulus M , the intermediate list has size L^2/M , and the complexity is $\max(L, L^2/M)$, not $\max(L, \tilde{L})$. A recursive variant (HGJ, Sec. 4.3) applies the decomposition to the complementary knapsack near $\alpha = 1/2$ and has thresholds $\alpha_0 \approx 0.312835$, $\alpha_1 \approx 0.458959$.

C. Generalised coefficient sets: BCJ

Becker, Coron and Joux [9] extend the coefficient set from $\{0, 1\}$ to $\{-1, 0, 1\}$: a 1 splits as $(0, 1)$ or $(1, 0)$, and a 0 as $(0, 0)$, $(1, -1)$, or $(-1, 1)$. Using three levels of decomposition with parameters α, β, γ :

$$N_\nu(1) = \frac{1}{4} + \alpha, \quad N_\nu(-1) = \alpha, \quad (3)$$

$$N_\kappa(1) = \frac{1}{8} + \frac{\alpha}{2} + \beta, \quad N_\kappa(-1) = \frac{\alpha}{2} + \beta, \quad (4)$$

$$N_\omega(1) = \frac{1}{16} + \frac{\alpha}{4} + \frac{\beta}{2} + \gamma, \quad N_\omega(-1) = \frac{\alpha}{4} + \frac{\beta}{2} + \gamma. \quad (5)$$

With moduli fixed by the single-decomposition condition, the list sizes are $L_\chi = \text{mult}(N_\chi) / \prod M$ and

$$T = \max\left(L_\omega, L_\omega^{1/2}, \frac{L_\omega^2}{M_\kappa}, L_\kappa, \frac{L_\kappa^2}{M_\nu}, L_\nu, L_\nu^2 \frac{M_\nu M_\kappa M_\omega}{2^n}\right). \quad (6)$$

Minimising gives $2^{0.291n}$ at $\alpha = 0.0267$, $\beta = 0.0168$, $\gamma = 0.0029$; at $\alpha = \beta = \gamma = 0$ it recovers 0.337. Bonnetain, Bricout, Schrottenloher and Shen [10] use four-valued representations to reach $2^{0.283n}$. These are *average-case* bounds for random hard knapsacks, resting on heuristic modular-distribution corollaries.

D. Reproduced exponents

We implemented the HGJ and BCJ models and minimised them numerically (Table III); the companion note gives the derivation. The results match the primary sources.

E. Pseudo-solutions and compatibility certificates

Increasing the number of coefficients per representation (e.g. allowing the value 2 or extra ± 1 s) enlarges the solution-pair space so that a smaller filtered list suffices, but creates *pseudo-solutions*: pairs that satisfy the sum/modular condition yet combine to an invalid coefficient vector. The algorithm must find a *compatible* pair, certified by a sampled set of indices [4], [11]. This “compatibility certificate” method is used for Equal Subset Sum [11] and underlies the worst-case results below.

TABLE III
REPRESENTATION EXPONENTS, REPRODUCED FROM [8], [9].

Quantity	Exponent
HGJ two-part ideal $D(1/2)$	0.3113
HGJ simple algorithm (best β)	0.337
BCJ $\alpha = \beta = \gamma = 0$ (recovers HGJ)	0.337
BCJ minimised	0.291

TABLE IV
WORST-CASE SUBSET BALANCING OVER COEFFICIENT SETS C [11].

C	Worst-case exponent
$[-d : d], d > 1$	$ C ^{0.5-\epsilon}$
$[\pm d], d > 2$	$ C ^{0.5-\epsilon}$
$[-1 : 1]$ (ESS)	1.7067^n
$[\pm 1]$ (Partition)	open
$\{\pm 2\}$	open
$\{0, 1\}$ (Subset Sum)	open

V. WORST-CASE REPRESENTATION: MIXING AND COEFFICIENT SHIFTING

Randolph and Węgrzycki [11] move the representation technique to the *worst case* for a wide range of coefficient sets.

a) Mixing dichotomies. For random instances, solution representations “mix well” (their residues mod M are spread). For worst-case instances, they prove dichotomies: if solution pairs do not mix well, the instance admits special structure (e.g. a small doubling constant, or exponentially many solutions) solvable by subsampling or a different algorithm; otherwise the generalised representation method applies.

b) Coefficient shifting. For coefficient sets without a 0, they factor $C = C_1 + C_2$ so that coefficients acquire several representations, e.g. $\{0, 1\} + \{-3, -2, 1, 2\} = [\pm 3]$, where ± 2 has two representations.

c) Results and open cases. Table IV lists the outcomes. The results leave exactly three coefficient sets open: $\{0, 1\}$ (Subset Sum), $[\pm 1]$ (Partition), and $\{\pm 2\}$. We independently verified (Section XIII) that these are precisely the sets admitting *no nontrivial subset factorisation*, so coefficient shifting has nothing to exploit.

VI. WHERE THE TARGET IS ALREADY ATTAINED: RELATED VARIANTS

a) Pigeonhole Equal Subset Sum. PESS (total sum $< 2^n - 1$, so a collision is guaranteed) is solved in $\mathcal{O}^*(2^{n/3})$ by a randomized algorithm (Zhang, 2024; Jin–Williams–Zhang, ESA 2025 [14]), improving Jin–Wu’s $\mathcal{O}^*(2^{0.4n})$ [13]. The key structural quantity is $F = \sum_t \max(0, \text{cnt}(t) - 1)$, the number of colliding subset-sum pairs. Small F forces the weights to be close to the geometric progression $1, 2, 4, \dots$, collapsing PESS to a small ESS instance on a few low-order elements; large F means many solutions, handleable by subsampling and mod- p dynamic programming. A polynomial-space variant runs in $\mathcal{O}^*(2^{2n/3})$ time. A FOCS 2026 line gives a deterministic dense algorithm $\text{poly}(n)M^{o(1)}$ ($M = 2^{o(n)}$), an average-case $\text{poly}(n)M^{1/4}$, and a lattice-based conditional lower bound.

b) Equal Subset Sum. Worst-case ESS is at $\mathcal{O}^*(1.6994^n)$ (Yamano–Shibuya, 2026) [12], improving 1.7067^n [11] and 1.7088^n (MNPW, ESA 2019); the baseline is $3^{n/2} = 1.7321^n$.

c) Either-Or Subset Sum. A hybrid returning either a subset-sum witness or a pair of equal-sum subsets is solvable in randomized worst-case $\mathcal{O}^*(2^{0.461n})$ (Randolph’s thesis [15]), the strongest concrete evidence toward breaking the meet-in-the-middle barrier for $\{0, 1\}$ itself.

d) Quantum algorithms. Bernstein, Jeffery, Lange and Meurer [16] solve worst-case subset sum in $\mathcal{O}^*(2^{n/3})$; Chukhin, Kulikov, Levitskii and Mihajlin [17] give $\mathcal{O}^*(2^{2n/7}) \approx 2^{0.2857n}$ via a worst-case quantum k -SUM algorithm with exponent $\Psi_k = 2k/7 + \mathcal{O}(1)$. Hence $1/3$ is not information-theoretically special; the classical obstruction is algorithmic.

e) Cardinality-constrained. Fixing the Hamming weight of the solution turns the problem into k -SUM, where meet-in-the-middle is $\mathcal{O}(n^{\lceil k/2 \rceil})$ and the k -SUM conjecture asserts this is essentially optimal. In the low-weight cryptanalytic regime, Esser and May (EUROCRYPT 2020) give a heuristic $2^{0.645n}$ algorithm (improving BCJ’s $2^{0.72n}$ random-subset-sum bound); it needs many solutions and is not worst-case.

VII. PSEUDOPOLYNOMIAL, DENSE, AND PROXIMITY ALGORITHMS

Table V collects the pseudopolynomial and dense results.

Any of these meets the $2^{n/3}$ target whenever $t \leq 2^{n/3}$; the hard regime is large values / low density.

a) Proximity and integer programming. Eisenbrand and Weismantel’s Steinitz-lemma proximity gives $(m\Delta)^{\mathcal{O}(m)}$ poly algorithms for integer programs; Randolph and Węgrzycki (ESA 2024) prove an exact equivalence between bounded-doubling subset sum and hyperplane-constrained binary ILP, recasting subset-sum hardness as ILP dimension/additive structure. For vanilla $\{0, 1\}$ ($m = 1$) this yields only a pseudopolynomial bound.

VIII. DISSECTION, GENERALISED BIRTHDAY, AND CRYPTANALYSIS

a) Generalised birthday. Wagner’s k -tree algorithm [18] solves the modular k -sum in $\mathcal{O}(k \cdot 2^{n/(1+\log k)})$; for $k = 4$ this is $2^{n/3}$. Lyubashevsky (2005) and Shallue (2008) apply it to Random Modular Subset Sum with modulus $M = 2^{n^\epsilon}$, giving $2^{\mathcal{O}(n^\epsilon / \log n)}$ bounds. These are average-case results for dense/many-solution instances; the representation technique is the low-density counterpart.

b) Lattice reduction. Lagarias and Odlyzko [19] solve almost all random knapsacks of density $d < 0.6463$ in polynomial time given an SVP oracle; Coster et al. extend this to $d < 0.9408$. BKZ-based attacks dominate the density- ≈ 1 regime. Joux and Węgrzycki (STACS 2026) improve the modular approach, roughly doubling the admissible density. Lattice methods are average-case/heuristic and orthogonal to the worst-case target.

TABLE V
PSEUDOPOLYNOMIAL AND DENSE ALGORITHMS FOR SUBSET SUM.

Algorithm	Time	Notes	Ref.
Bellman	$\mathcal{O}(nt)$	classic DP	1957
Koiliaris–Xu	$\tilde{\mathcal{O}}(t\sqrt{n})$	deterministic	[20]
Bringmann	$\tilde{\mathcal{O}}(n+t)$	randomized	[21]
Jin–Wu	$\tilde{\mathcal{O}}(n+t)$	simpler randomized	SOSA’19
Bringmann–Nakos	$\tilde{\mathcal{O}}(S(X,t) ^{4/3})$	output-sensitive	STOC’20
Chen–Lian–Mao–Zhang	$\tilde{\mathcal{O}}(n + \sqrt{a_{\max}t})$		[22]
Bringmann–Fischer–Nakos	$\tilde{\mathcal{O}}(S(X,t) \sqrt{n})$	beats Bellman in all regimes	[23]
Chan	$\tilde{\mathcal{O}}(t)$	deterministic	SODA’26
Bringmann–Wellnitz	$\tilde{\mathcal{O}}(n)$	dense decision	[24]
Chen–Mao–Zhang	$\tilde{\mathcal{O}}(n)$	dense search with witness	[25]

TABLE VI
CONDITIONAL LOWER BOUNDS AND THEIR ASSUMPTIONS.

Bound	Assumption
No $2^{o(n)}$ exact algorithm	ETH
No $\tilde{\mathcal{O}}(T^{1-\epsilon}2^{o(n)})$	SETH (Abboud et al.)
No $\mathcal{O}(n^{\lceil k/2 \rceil - \epsilon})$ for k -SUM	k -SUM conjecture
No truly subquadratic 3SUM	3SUM conjecture
No truly subquadratic (min,+) convolution	MinConv hypothesis
No $2^{(1/2-\epsilon)n}$ for subset sum	Meet-in-the-Middle conjecture (unproven)

IX. ADDITIVE COMBINATORICS AND THE DENSITY DICHOTOMY

Classical results (Sárközy, Alon–Kleitman, Szemerédi–Vu) describe how long arithmetic progressions and structured sumsets force dense subset sums to behave well. A recurring theme is a *dichotomy*: either the instance has additive structure (small sumset, small doubling, few representations) and is easy, or it has many representations and the representation/subsampling machinery applies. The sub-1/2 worst-case results [11] are built on such dichotomies; the coefficient sets without a 0 resist them. We probed two folklore hypotheses with small experiments: dissociativity ($F = 0$) does not make an instance hard (representations remain modularly spread), and pseudo-solution load is concentrated in the *easy* collision-heavy families, not in the hard-looking dissociated ones.

X. LOWER BOUNDS AND BARRIERS

No established hypothesis rules out a worst-case exponent below 1/2: ETH only forbids $2^{o(n)}$; SETH pins the pseudopolynomial factor $T^{1-\epsilon}2^{o(n)}$; 3SUM/ k -SUM/MinConv are not known to transfer. An $\mathcal{O}^*(2^{n/3})$ algorithm would contradict only the unproven Meet-in-the-Middle conjecture, which Either-Or Subset Sum already weakens.

XI. LANDSCAPE

XII. CHRONOLOGY

XIII. THE FRONTIER AND OPEN PROBLEMS

We independently searched sumset factorisations $C = C_1 + C_2$ and found that the three open coefficient sets are exactly those with no nontrivial factorisation (every coefficient keeps one representation), whereas $\{-1, 0, 1\}$ and $\{\pm 3\}$ acquire multiples (Table IX).

a) Open problems.:

- 1) Is there an $\mathcal{O}^*(2^{(1/2-\epsilon)n})$ worst-case algorithm for $\{0, 1\}$ subset sum? The equivalent open cases are Partition and $\{\pm 2\}$.
- 2) Prove or refute the Meet-in-the-Middle conjecture; Either-Or Subset Sum is the strongest evidence against it.
- 3) Close the average-case/worst-case gap for $\{0, 1\}$: find a worst-case mechanism that does not need a sumset factorisation, or a conditional lower bound.
- 4) Optimise space constants across the worst-case results ($\mathcal{O}^*(2^{0.246n})$ space; poly-space exponents).

XIV. REPRODUCIBILITY

The exponents in Table III and the coefficient-shifting search are produced by dependency-free Python scripts (`representation_exponents.py`, `coefficient_shifting.py`) whose Markdown outputs and provenance live under `docs/analysis/`. A companion baselines note covers correctness, implementations, and measurements of meet-in-the-middle, Schroeppe–Shamir, and the signed DP, with a Lean correctness layer.

REFERENCES

- [1] E. Horowitz and S. Sahni, “Computing partitions with applications to the knapsack problem,” *J. ACM* 21(2):277–292, 1974.
- [2] R. Schroeppe and A. Shamir, “A $T = O(2^{n/2})$, $S = O(2^{n/4})$ algorithm for certain NP-complete problems,” *SIAM J. Comput.* 10(3):456–464, 1981.
- [3] X. Chen, Y. Jin, T. Randolph, R. A. Servedio, “Subset sum in time $2^{n/2}/\text{poly}(n)$,” *RANDOM* 2023; arXiv:2301.07134.
- [4] J. Nederlof and K. Węgrzycki, “Improving Schroeppe and Shamir’s algorithm for subset sum via orthogonal vectors,” *STOC* 2021; arXiv:2010.08576.
- [5] E. Belova, I. Chukhin, A. Kulikov, I. Mihajlin, “Improved space bounds for subset sum,” *ESA* 2024; arXiv:2402.13170.

TABLE VII
EXACT SUBSET SUM AND BALANCING: ALGORITHMS AND BOUNDS. WC = WORST CASE; AVG = AVERAGE CASE / HEURISTIC.

Algorithm / result	Time	Space	Regime	Setting / notes	Ref.
Horowitz–Sahni	$\mathcal{O}^*(2^{n/2})$	$\mathcal{O}^*(2^{n/2})$	WC	deterministic, witness	[1]
Schroeppel–Shamir	$\mathcal{O}^*(2^{n/2})$	$\mathcal{O}^*(2^{n/4})$	WC	deterministic	[2]
Chen et al.	$2^{n/2}/n^{0.5023}$	$\mathcal{O}^*(2^{n/4})$	WC	word/circuit RAM	[3]
Nederlof–Węgrzycki	$\mathcal{O}^*(2^{0.5n})$	$\mathcal{O}^*(2^{0.249999n})$	WC	random, via OV	[4]
Belova et al.	$\mathcal{O}^*(2^{0.5n})$	$\mathcal{O}^*(2^{0.246n})$	WC	lower space	[5]
Bansal et al.	$\mathcal{O}^*(2^{0.86n})$	poly	WC	algebraization	[6]
Lokshtanov–Nederlof	$\tilde{\mathcal{O}}(n^3 t \log t)$	$\tilde{\mathcal{O}}(n^2)$	pseudo-poly	subset sum	[7]
Howgrave-Graham–Joux	$2^{0.337n}$	$2^{0.256n}$	avg	hard knapsack	[8]
Becker–Coron–Joux	$2^{0.291n}$	—	avg	hard knapsack	[9]
Bonnetain et al.	$2^{0.283n}$	—	avg	4-valued repr.	[10]
Randolph–Węgrzycki	$ C ^{(0.5-\epsilon)n}$	—	WC	most constant C	[11]
Equal Subset Sum	1.6994^n	—	WC	2026	[12]
Pigeonhole ESS	$\mathcal{O}^*(2^{n/3})$	—	promise	randomized	[14]
Either-Or Subset Sum	$\mathcal{O}^*(2^{0.461n})$	—	WC	randomized	[15]
Quantum (BJLM)	$\mathcal{O}^*(2^{n/3})$	—	WC	quantum	[16]
Quantum (Chukhin et al.)	$\mathcal{O}^*(2^{2n/7})$	—	WC	quantum	[17]
Wagner k -tree ($k = 4$)	$2^{n/3}$	$2^{n/3}$	avg	modular k -sum	[18]
Lattice (Lagarias–Odlyzko)	poly	poly	avg	$d < 0.6463$, SVP oracle	[19]

TABLE VIII
SELECTED MILESTONES.

Year	Milestone
1957	Bellman DP
1974	Horowitz–Sahni MITM
1981	Schroeppel–Shamir
1985	Lagarias–Odlyzko lattice
1991	Coster et al. (density 0.94)
2002	Wagner generalised birthday
2010	HGJ representation; Lokshtanov–Nederlof
2011	Becker–Coron–Joux 0.291
2013	Bernstein et al. quantum $2^{n/3}$
2017	Bringmann; Koiliaris–Xu; Bansal et al.
2019	MNPW ESS 1.7088
2020	Bonnetain et al. 0.283
2021	Nederlof–Węgrzycki (OV)
2023	Chen et al. $2^{n/2}/\text{poly}$
2024	Belova et al.; Jin–Wu; Zhang PESS
2025	Jin–Williams–Zhang PESS; Randolph–Węgrzycki; BFN; CMZ
2026	Yamano–Shibuya; Chukhin et al.; Chan; Joux–Węgrzycki

TABLE IX
COEFFICIENT-SHIFTING SEARCH OVER $C_1, C_2 \subseteq [-3, 3]$.

C	nontrivial factorisations	max representations
$\{0, 1\}$	0	1
$\{\pm 1\}$	0	1
$\{\pm 2\}$	0	1
$\{-1, 0, 1\}$	6	2
$\{\pm 3\}$	4	2
$\{-2, \dots, 2\}$	39	3

- [8] N. Howgrave-Graham and A. Joux, “New generic algorithms for hard knapsacks,” EUROCRYPT 2010; IACR ePrint 2010/189.
- [9] A. Becker, J.-S. Coron, A. Joux, “Improved generic algorithms for hard knapsacks,” EUROCRYPT 2011; IACR ePrint 2011/474.
- [10] X. Bonnetain, R. Bricout, A. Schrottenloher, Y. Shen, “Improved classical and quantum algorithms for subset-sum,” ASIACRYPT 2020; ePrint 2020/168.
- [11] T. Randolph and K. Węgrzycki, “Beating meet-in-the-middle for subset balancing problems,” arXiv:2511.10823, 2025.
- [12] Yamano and Shibuya, “Faster exact algorithms for Equal-Subset-Sum,” arXiv:2607.09289, 2026.
- [13] C. Jin and X. Wu, “A faster algorithm for Pigeonhole Equal Sums,” ICALP 2024; arXiv:2403.19117.
- [14] C. Jin, R. Williams, S. Zhang, “New algorithms for Pigeonhole Equal Subset Sum,” ESA 2025, LIPIcs 351:86.
- [15] T. Randolph, “Exact and parameterized algorithms for subset sum problems,” PhD thesis, Columbia University, 2024.
- [16] D. Bernstein, S. Jeffery, T. Lange, A. Meurer, “Quantum algorithms for the subset-sum problem,” PQCrypto 2013.
- [17] I. Chukhin, A. Kulikov, V. Levitskii, I. Mihajlin, “Improved quantum algorithms for subset sum and k -SUM,” arXiv:2608.07309, 2026.
- [18] D. Wagner, “A generalized birthday problem,” CRYPTO 2002.
- [19] J. Lagarias and A. Odlyzko, “Solving low-density subset sum problems,” J. ACM 32(1):229–246, 1985.
- [20] K. Koiliaris and C. Xu, “A faster pseudopolynomial time algorithm for subset sum,” SODA 2017.
- [21] K. Bringmann, “A near-linear pseudopolynomial time algorithm for subset sum,” SODA 2017.
- [22] L. Chen, J. Lian, Y. Mao, G. Zhang, “An improved pseudopolynomial time algorithm for subset sum,” FOCS 2024.
- [23] K. Bringmann, N. Fischer, V. Nakos, “Beating Bellman’s algorithm for subset sum,” SODA 2025; arXiv:2410.21942.
- [24] K. Bringmann and P. Wellnitz, “On near-linear-time algorithms for dense subset sum,” SODA 2021; arXiv:2010.09096.
- [25] L. Chen, Y. Mao, G. Zhang, “Long arithmetic progressions in sumsets and subset sums,” STOC 2025; arXiv:2503.19299.

- [6] N. Bansal, S. Garg, J. Nederlof, N. Vyas, “Faster space-efficient algorithms for subset sum and knapsack,” FOCS 2017.
- [7] D. Lokshtanov and J. Nederlof, “Saving space by algebraization,” STOC 2010.